

Array Index Formulas (1D, 2D, 3D, n-D) - Universal & Exam-Friendly

1 1D Array

Array: $A[L..U]$

Formula:

$$\text{Address}(A[i]) = \text{Base} + (i - L) * \text{Width}$$

Explanation: - Base = base address of array in memory - L = lower bound of array (starting index, e.g., 0 or 1) - U = upper bound of array (ending index) - Width = size of each element in bytes - i = index of element to access

Example: $A[1..10]$, Base=100, Width=1 $\rightarrow A[5] = 100 + (5-1)*1 = 104$

2 2D Array

Array: $A[L1..U1][L2..U2]$

- Rows = $U1-L1+1$ - Cols = $U2-L2+1$

Explanation of terms: - L1 = lower bound of rows (starting row index) - U1 = upper bound of rows (ending row index) - L2 = lower bound of columns (starting column index) - U2 = upper bound of columns (ending column index) - i = row index to access - j = column index to access - Width = size of one element in bytes

a) Row-Major Order (C/C++)

$$\text{Address}(A[i][j]) = \text{Base} + [(i-L1) * \text{Cols} + (j-L2)] * \text{Width}$$

b) Column-Major Order (Fortran)

$$\text{Address}(A[i][j]) = \text{Base} + [(j-L2) * \text{Rows} + (i-L1)] * \text{Width}$$

Example: $A[1..3][1..4]$, Base=100, Width=1, find $A[2][3]$ - Rows = 3, Cols = 4 - Row-major: $100 + [(2-1)*4 + (3-1)*1] = 100 + [4+2] = 106$ - Column-major: $100 + [(3-1)*3 + (2-1)*1] = 100 + [6+1] = 107$

3 3D Array

Array: $A[L1..U1][L2..U2][L3..U3]$

- Sizes: $X = U1-L1+1$, $Y = U2-L2+1$, $Z = U3-L3+1$

a) Row-Major Order

$$\text{Address}(A[i][j][k]) = \text{Base} + [(i-L1)*(Y*Z) + (j-L2)*Z + (k-L3)] * \text{Width}$$

b) Column-Major Order

$$\text{Address}(A[i][j][k]) = \text{Base} + [(k-L3)*(X*Y) + (j-L2)*X + (i-L1)] * \text{Width}$$

Example: $A[1..2][1..3][1..4]$, Base=100, Width=1, find $A[2][3][4]$ - X=2, Y=3, Z=4 - Row-major: $100 + [(2-1)(34) + (3-1)4 + (4-1)]1 = 100 + (12 + 8 + 3) = 123$ - Column-major: $100 + [(4-1)(23) + (3-1)2 + (2-1)]1 = 100 + (18 + 4 + 1) = 123$

4 n-Dimensional Array

Array: $A[L1..U1][L2..U2]...[Ln..Un]$

- $D_k = U_k - L_k + 1$ (size of k-th dimension)

a) Row-Major Order

$$\text{Address}(A[i1][i2]...[in]) = \text{Base} + [(i1-L1)*(D2*D3*...*Dn) + (i2-L2)*(D3*...*Dn) + ... + (in-Ln)] * \text{Width}$$

b) Column-Major Order

$$\text{Address}(A[i1][i2]...[in]) = \text{Base} + [(in-Ln)*(D1*D2*...*D(n-1)) + (i(n-1)-L(n-1))*(D1*...*D(n-2)) + ... + (i1-L1)] * \text{Width}$$

Explanation of terms: - L_i = lower bound of i-th dimension - U_i = upper bound of i-th dimension - D_k = size of k-th dimension = $U_k - L_k + 1$ - $i1, i2, ... in$ = indices of element to access - Width = size of one element in bytes

Example (4D simplified): $A[1..2][1..2][1..3][1..3]$, Base=200, Width=1, find $A[2][1][3][2]$ - Compute sizes: $D1=2, D2=2, D3=3, D4=3$ - Row-major: Address = $200 + [(2-1)(233) + (1-1)(33) + (3-1)3 + (2-1)]1 = 200 + [18 + 0 + 6 + 1] = 225$

5 Quick Memorization Tips

- 1D $\rightarrow$ Base + (index - lower bound) * width
- 2D $\rightarrow$ Row-major: multiply row index by #columns; Column-major: multiply column index by #rows
- 3D $\rightarrow$ Row-major: row(col*depth) + col*depth + depth index; Column-major: multiply from last dimension backward

4. n-D $\rightarrow$ Row-major: multiply current index by product of remaining dimensions; Column-major: multiply from last dimension backward
5. Always subtract **lower bounds** in formula